PYTHON ACTIVITY-BASED LEARNING

10 Practical Programs — Complete Activity Report

Problem Statements • Objectives • Assumptions • Algorithms • Program Screenshots • Output Screenshots

Prepared using the complete contents of the supplied second PDF

Contents

1. Attendance Recovery Computation
2. Hospital Billing and Patient Classification
3. Library Borrowing and Fine Reconciliation
4. Multi-Product Shopping Budget Constraint
5. Multi-Stage Scholarship
6. Password Policy Validation Engine
7. Progressive Electricity Billing
8. Smart Canteen Order and Stock Management
9. Smart Parking Fee Management
10. University Scholarship Allocation

Each activity includes a written explanation, the original Python code screenshot, and the original output screenshot supplied in the source PDF.

1. Attendance Recovery Computation

1. Problem Statement

Calculate a student's current attendance percentage and determine how many upcoming classes must be attended consecutively to reach the required 75% attendance.

2. Objectives

• Read classes attended and classes conducted.

• Calculate the current attendance percentage.

• If attendance is below 75%, calculate the number of future classes needed.

• Display the number of classes to attend or confirm that the required attendance is already met.

3. Assumptions

• The target attendance is 75%.

• Each future class is assumed to be attended.

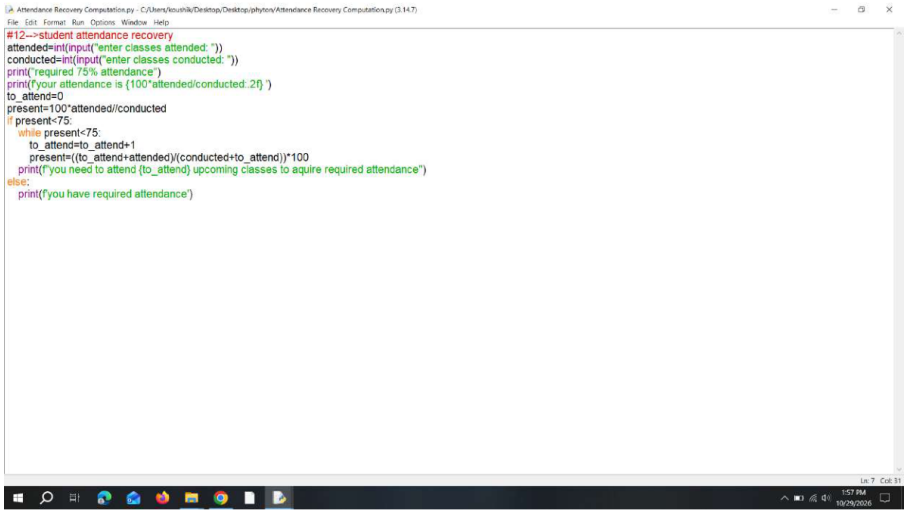
• The program uses the values entered by the user.

4. Algorithm

Read attended and conducted classes → calculate percentage → compare with 75% → calculate extra classes if needed → display result.

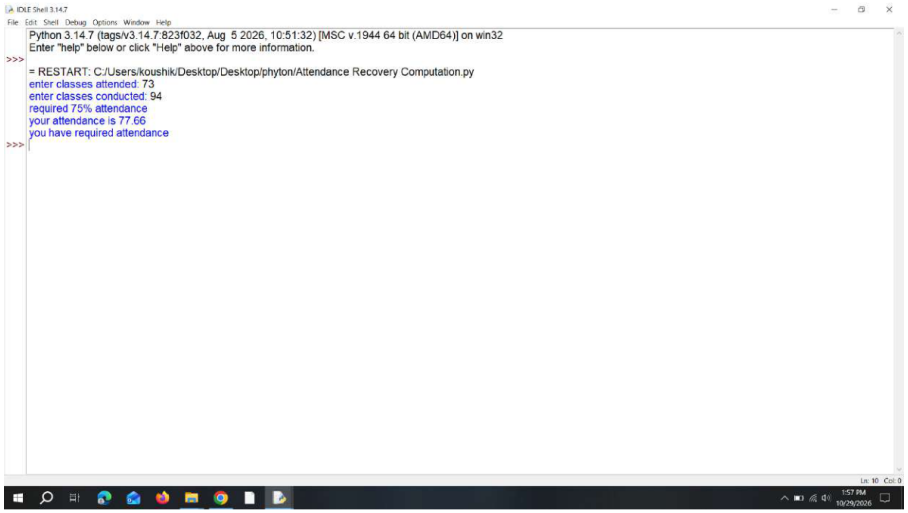
Activity 1: Attendance Recovery Computation

6. Original Python Program Screenshot



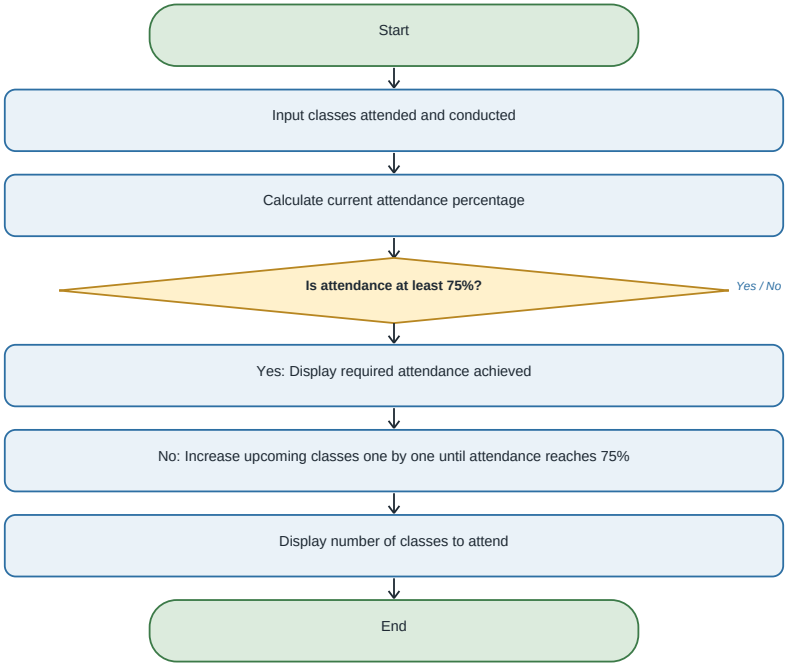
Activity 1: Attendance Recovery Computation

7. Original Output Screenshot



Activity 1: Flowchart

Attendance Recovery Computation



Activity 1: Pseudocode

Attendance Recovery Computation

```
START
INPUT attended, conducted
SET required = 75
SET present = (attended / conducted) × 100
IF present >= required THEN
    DISPLAY “You have required attendance”
ELSE
    SET to_attend = 0
    WHILE present < required
        to_attend = to_attend + 1
        present = ((attended + to_attend) / (conducted + to_attend)) × 100
    END WHILE
    DISPLAY to_attend
END IF
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 1: Test Cases

Attendance Recovery Computation

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	73, 94	77.66%; required attendance already achieved
2	60, 100	Attend 60 upcoming classes
3	70, 100	Attend 20 upcoming classes
4	74, 100	Attend 4 upcoming classes
5	75, 100	No extra classes required
6	80, 100	No extra classes required
7	90, 100	No extra classes required
8	45, 60	No extra classes required (75%)
9	30, 50	Attend 30 upcoming classes to reach 75%
10	0, 100	Attend 300 upcoming classes

Note: Boundary values are included to check conditions and decision branches.

Activity 1: Conclusion

Attendance Recovery Computation

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

2. Hospital Billing and Patient Classification

1. Problem Statement

Calculate patient bills from consultation, diagnostic, and treatment charges, apply a concession based on the bill amount, and classify each patient's final bill.

2. Objectives

• Read the number of patients and each patient's details.

• Add consultation, diagnostic, and treatment charges.

• Apply the concession according to the total charge.

• Calculate the final bill and assign a billing category.

• Display patient-wise billing details and the number of patients processed.

3. Assumptions

• The program processes multiple patients.

• The concession is based on the total charges as shown in the supplied program.

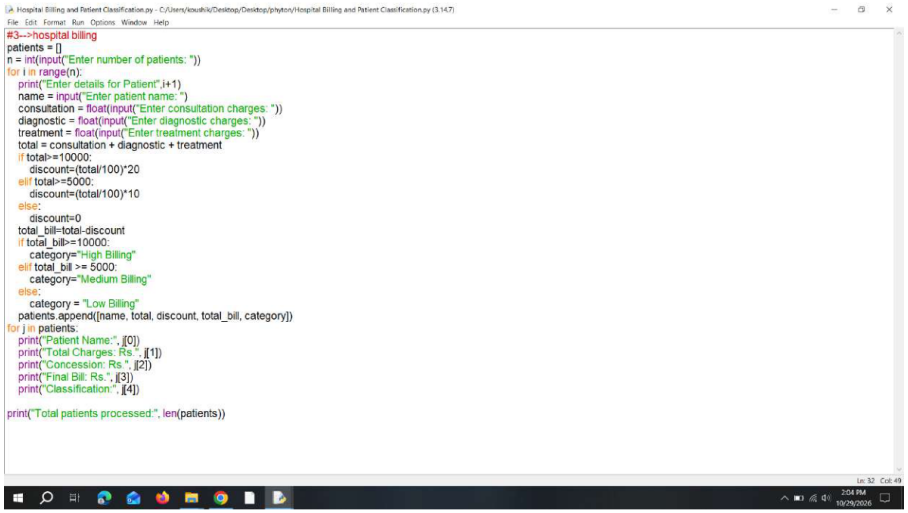
• Billing categories are determined from the final bill.

4. Algorithm

Read patient details → add charges → calculate concession → calculate final bill → classify bill → display patient summary.

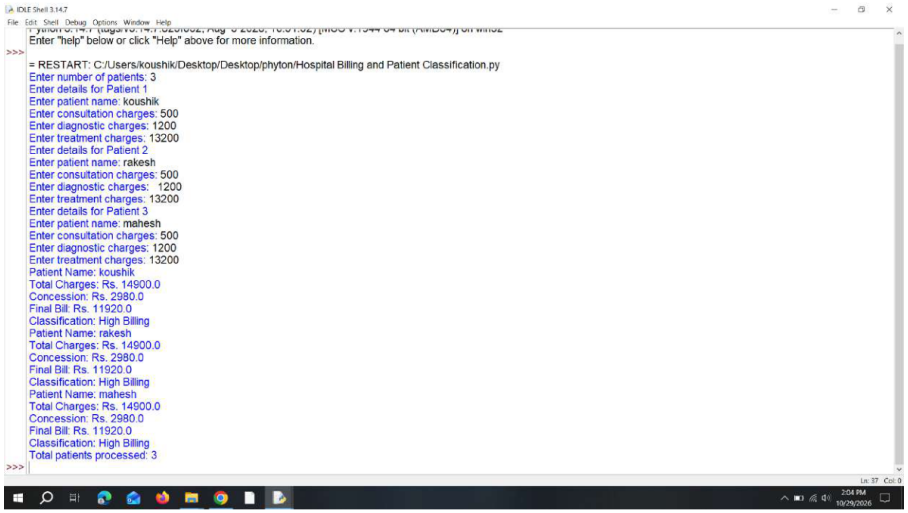
Activity 2: Hospital Billing and Patient Classification

6. Original Python Program Screenshot



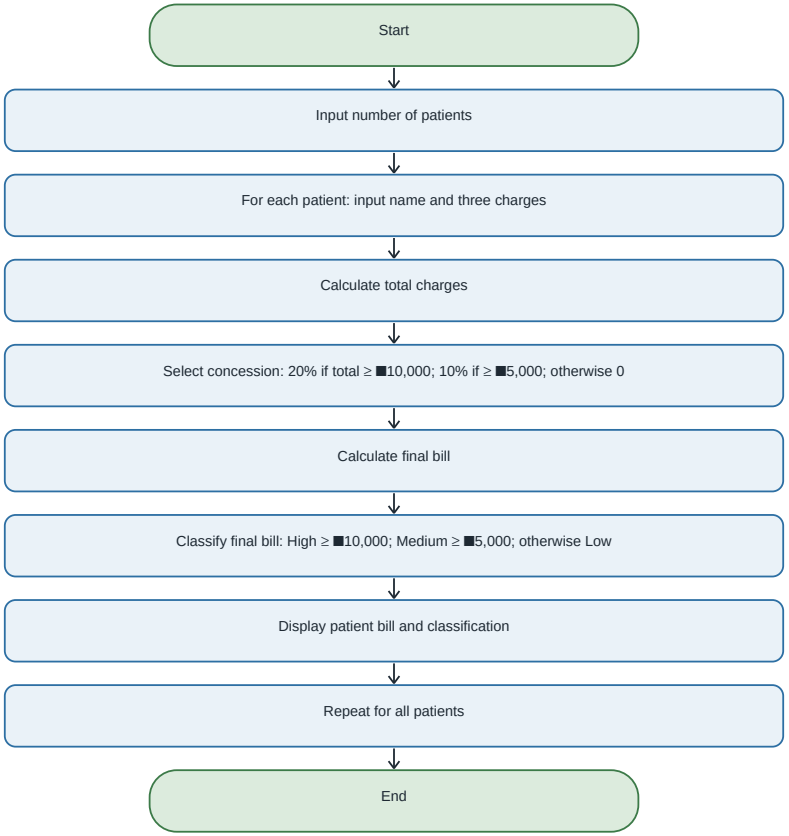
Activity 2: Hospital Billing and Patient Classification

7. Original Output Screenshot



Activity 2: Flowchart

Hospital Billing and Patient Classification



Activity 2: Pseudocode

Hospital Billing and Patient Classification

```
START
INPUT n
FOR each patient from 1 to n
    INPUT name, consultation, diagnostic, treatment
    total = consultation + diagnostic + treatment
    IF total >= 10000 THEN discount = total × 0.20
    ELSE IF total >= 5000 THEN discount = total × 0.10
    ELSE discount = 0
    final_bill = total - discount
    IF final_bill >= 10000 THEN category = “High Billing”
    ELSE IF final_bill >= 5000 THEN category = “Medium Billing”
    ELSE category = “Low Billing”
    DISPLAY name, total, discount, final_bill, category
END FOR
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 2: Test Cases

Hospital Billing and Patient Classification

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	Charges 2,000 + 1,000 + 1,000	Total ■4,000; discount ■0; final ■4,000; Low
2	2,000 + 2,000 + 1,000	Total ■5,000; discount ■500; final ■4,500; Low
3	2,000 + 2,000 + 1,000.01	Total ■5,000+; 10% discount; Low if final < ■5,000
4	5,000 + 3,000 + 2,000	Total ■10,000; discount ■2,000; final ■8,000; Medium
5	8,000 + 2,000 + 1,000	Total ■11,000; discount ■2,200; final ■8,800; Medium
6	10,000 + 5,000 + 5,000	Total ■20,000; discount ■4,000; final ■16,000; High
7	0 + 0 + 0	Total ■0; discount ■0; final ■0; Low
8	500 + 500 + 500	Total ■1,500; discount ■0; final ■1,500; Low
9	4,000 + 1,000 + 0	Total ■5,000; discount ■500; final ■4,500; Low
10	Patient count = 2; both totals ■6,000	Both processed; each receives 10% concession

Note: Boundary values are included to check conditions and decision branches.

Activity 2: Conclusion

Hospital Billing and Patient Classification

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

3. Library Borrowing and Fine Reconciliation

1. Problem Statement

Manage book borrowing and returns using a list of available books, and calculate a fine based on the number of days a returned book was borrowed.

2. Objectives

• Display the available library actions.

• Accept a borrow or return request.

• Check whether a requested book is available or valid.

• Update the book list for borrowing or returning.

• Calculate and display the fine for a returned book.

3. Assumptions

• The program uses a predefined list of valid books.

• The fine calculation is based on the borrowing duration in the supplied code.

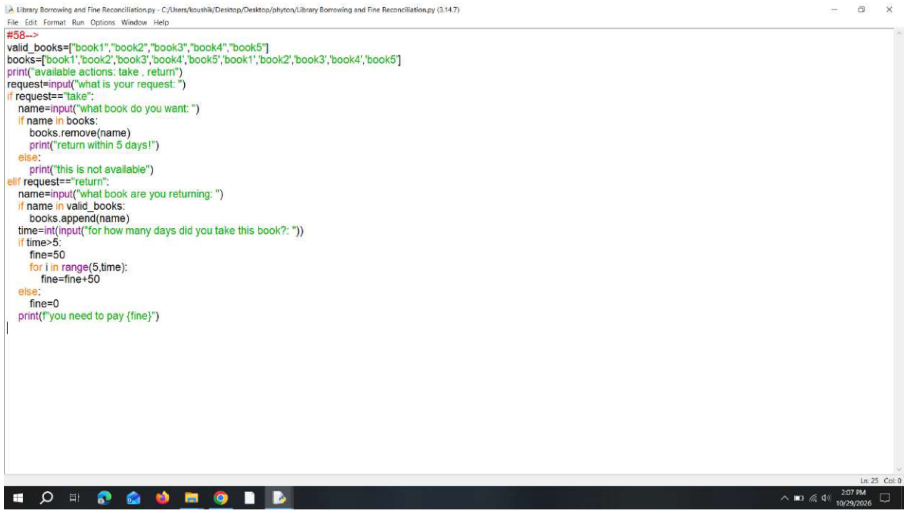
• This is a simple classroom library simulation.

4. Algorithm

Show actions → read request → check book → update book list → calculate fine when returned → display result.

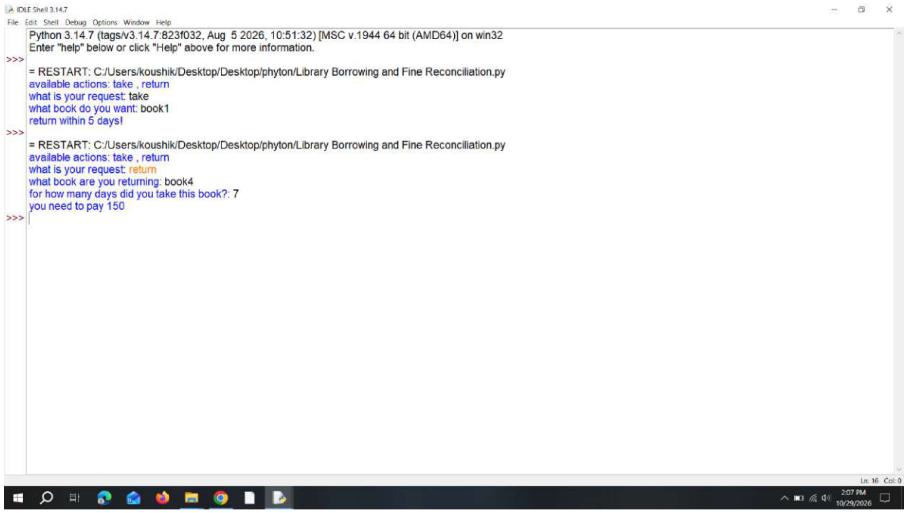
Activity 3: Library Borrowing and Fine Reconciliation

6. Original Python Program Screenshot



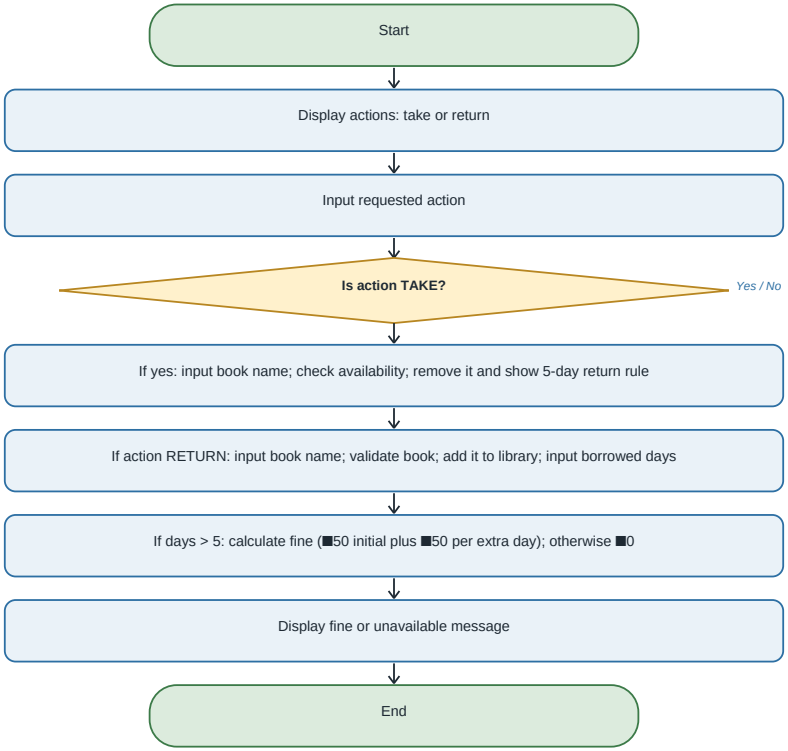
Activity 3: Library Borrowing and Fine Reconciliation

7. Original Output Screenshot



Activity 3: Flowchart

Library Borrowing and Fine Reconciliation



Activity 3: Pseudocode

Library Borrowing and Fine Reconciliation

```
START
SET valid_books and available_books
DISPLAY “take, return”
INPUT request
IF request = “take” THEN
    INPUT book_name
    IF book_name is in available_books THEN
        REMOVE book_name from available_books
        DISPLAY return within 5 days
    ELSE DISPLAY not available
ELSE IF request = “return” THEN
    INPUT book_name
    IF book_name is valid THEN
        ADD book_name to available_books
        INPUT days
        IF days > 5 THEN
            fine = 50 + 50 × (days - 6)
        ELSE fine = 0
        DISPLAY fine
    ELSE DISPLAY invalid book
ELSE DISPLAY invalid request
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 3: Test Cases

Library Borrowing and Fine Reconciliation

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	take; book1 (available)	Book removed from available list; return within 5 days
2	take; book not in stock	Display “this is not available”
3	return; book1; 3 days	Fine ■0
4	return; book1; 5 days	Fine ■0
5	return; book1; 6 days	Fine ■100 (■50 plus one loop increment)
6	return; book1; 7 days	Fine ■150 (■50 plus two loop increments)
7	return; book1; 8 days	Fine ■200 (■50 plus three loop increments)
8	return; invalid book name	Display unavailable/invalid book
9	request = borrow	No matching action; no valid operation
10	return; book2; 10 days	Fine increases with days beyond 5

Note: Boundary values are included to check conditions and decision branches.

Activity 3: Conclusion

Library Borrowing and Fine Reconciliation

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

4. Multi-Product Shopping Budget Constraint

1. Problem Statement

Calculate the total cost of multiple products and check whether the shopping bill is within the user's budget.

2. Objectives

• Read the shopping budget and number of products.

• Read each product name, price, and quantity.

• Calculate each product cost and add it to the total.

• Display product-wise costs and the total bill.

• Compare the total bill with the budget and show any excess.

3. Assumptions

• Prices are entered by the user.

• The total is calculated as price multiplied by quantity.

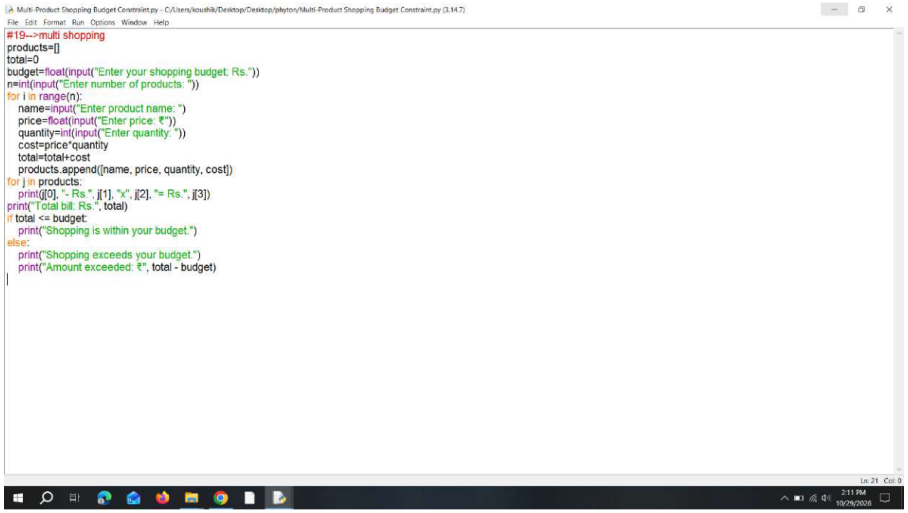
• The program reports budget status; it does not remove products automatically.

4. Algorithm

Read budget and product count → collect product details → calculate product costs → calculate total → compare with budget.

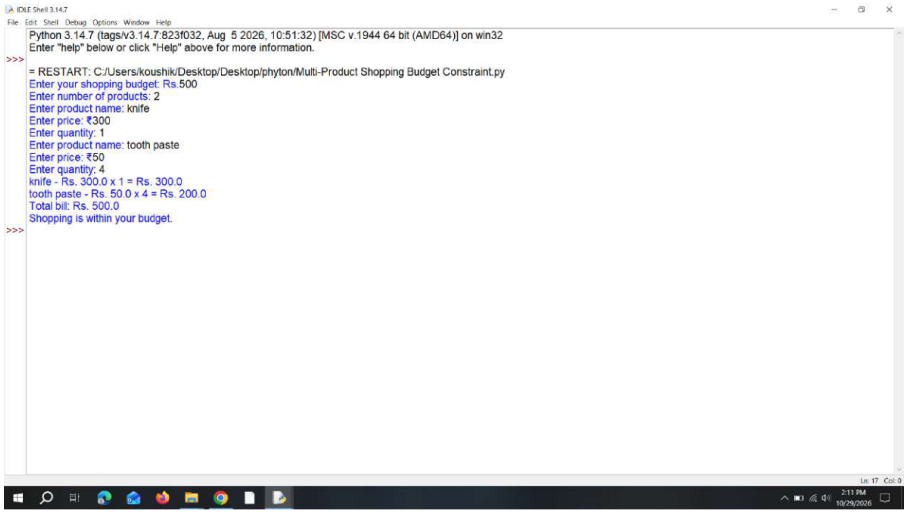
Activity 4: Multi-Product Shopping Budget Constraint

6. Original Python Program Screenshot



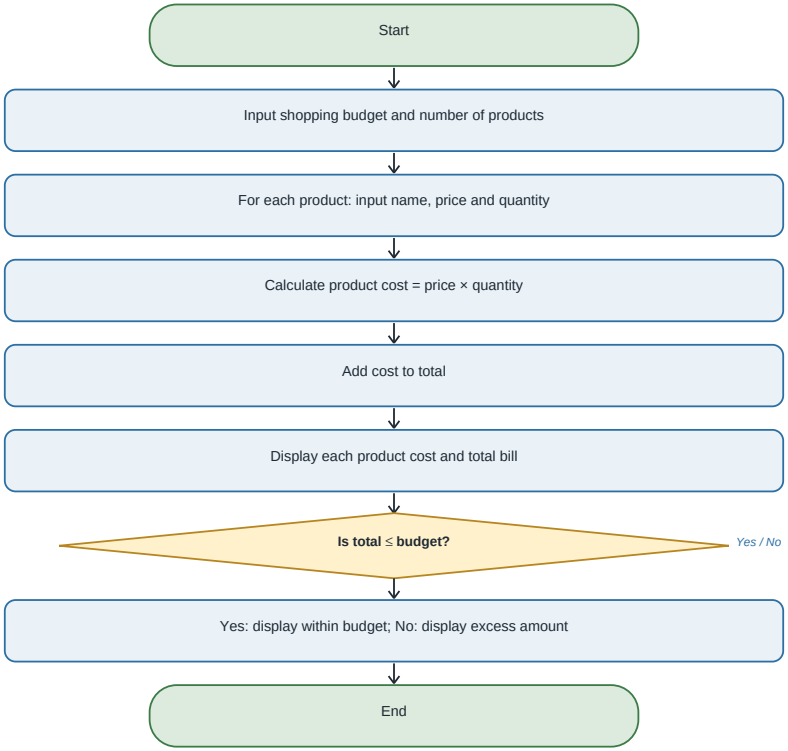
Activity 4: Multi-Product Shopping Budget Constraint

7. Original Output Screenshot



Activity 4: Flowchart

Multi-Product Shopping Budget Constraint



Activity 4: Pseudocode

Multi-Product Shopping Budget Constraint

```
START
INPUT budget, n
SET total = 0
FOR each product
    INPUT name, price, quantity
    cost = price × quantity
    total = total + cost
    STORE product details
END FOR
DISPLAY product details and total
IF total <= budget THEN
    DISPLAY “Shopping is within your budget”
ELSE
    DISPLAY “Shopping exceeds your budget”
    DISPLAY total - budget
END IF
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 4: Test Cases

Multi-Product Shopping Budget Constraint

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	Budget ■500; ■300 × 1; ■50 × 4	Total ■500; within budget
2	Budget ■400; ■300 × 1; ■50 × 4	Total ■500; exceeds by ■100
3	Budget ■1,000; one item ■1,000 × 1	Total ■1,000; within budget
4	Budget ■100; item ■150 × 1	Exceeds by ■50
5	Budget ■200; item ■20 × 0	Total ■0; within budget
6	Budget ■500; item ■100 × 5	Total ■500; within budget
7	Budget ■500; item ■100 × 6	Total ■600; exceeds by ■100
8	Budget ■0; item ■0 × 2	Total ■0; within budget
9	Budget ■1,000; items ■100 × 2 and ■200 × 3	Total ■800; within budget
10	Budget ■700; items ■100 × 2 and ■200 × 3	Total ■800; exceeds by ■100

Note: Boundary values are included to check conditions and decision branches.

Activity 4: Conclusion

Multi-Product Shopping Budget Constraint

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

5. Multi-Stage Scholarship

1. Problem Statement

Collect a student's name, marks percentage, attendance percentage, and income, then determine the scholarship category using the conditions in the supplied program.

2. Objectives

• Read the student's personal and academic details.

• Check the marks and attendance conditions.

• Check the income condition.

• Select the applicable scholarship category.

• Display the category for which the student can apply.

3. Assumptions

• Eligibility conditions are those written in the supplied program.

• The program uses a single student's entered details.

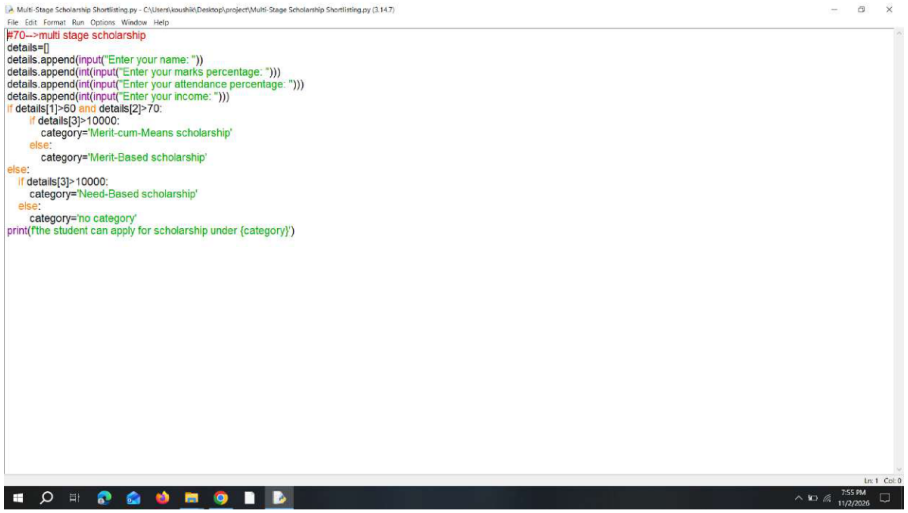
• The output identifies a category rather than calculating a scholarship amount.

4. Algorithm

Read student details → check marks and attendance → check income → choose scholarship category → display result.

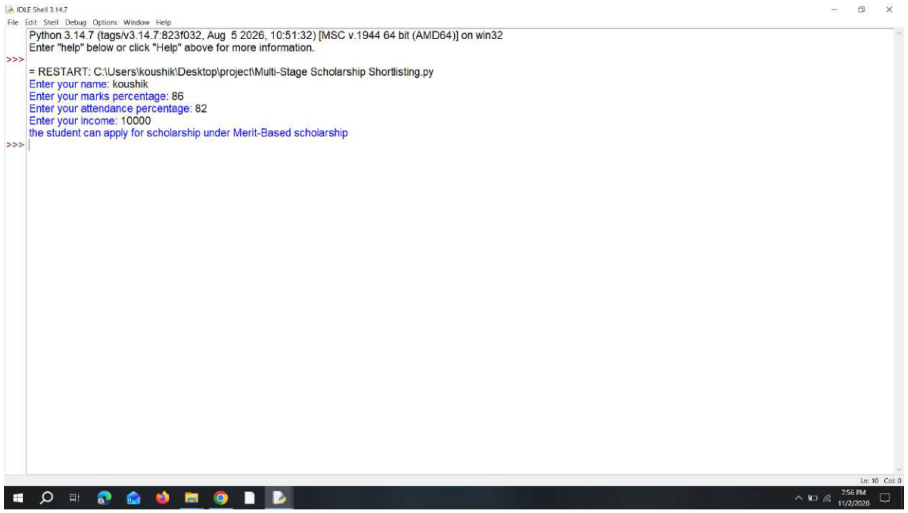
Activity 5: Multi-Stage Scholarship

6. Original Python Program Screenshot



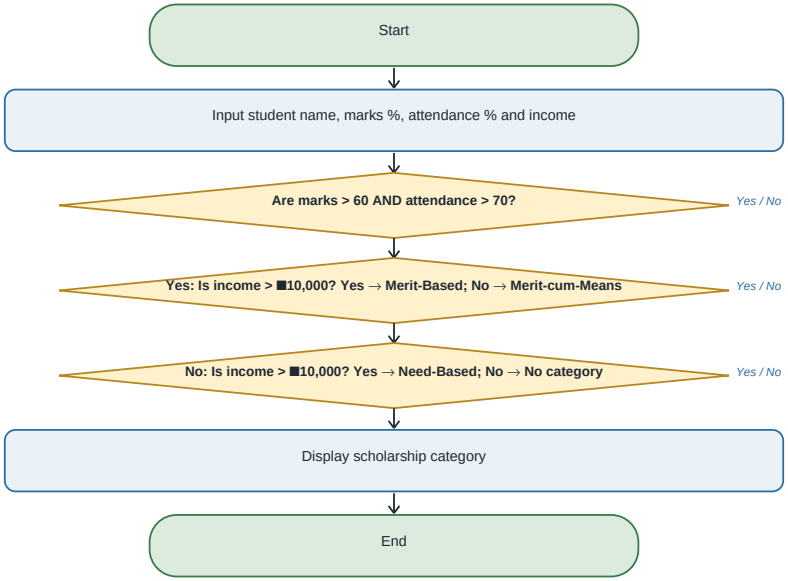
Activity 5: Multi-Stage Scholarship

7. Original Output Screenshot



Activity 5: Flowchart

Multi-Stage Scholarship Shortlisting



Activity 5: Pseudocode

Multi-Stage Scholarship Shortlisting

```
START
INPUT name, marks, attendance, income
IF marks > 60 AND attendance > 70 THEN
    IF income > 10000 THEN category = “Merit-Based scholarship”
    ELSE category = “Merit-cum-Means scholarship”
ELSE
    IF income > 10000 THEN category = “Need-Based scholarship”
    ELSE category = “No category”
DISPLAY name and category
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 5: Test Cases

Multi-Stage Scholarship Shortlisting

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	Marks 86; attendance 82; income ■10,000	Merit-cum-Means scholarship
2	Marks 86; attendance 82; income ■10,001	Merit-Based scholarship
3	Marks 60; attendance 80; income ■20,000	Need-Based scholarship
4	Marks 70; attendance 70; income ■20,000	Need-Based scholarship
5	Marks 60; attendance 70; income ■10,000	No category
6	Marks 90; attendance 90; income ■5,000	Merit-cum-Means scholarship
7	Marks 50; attendance 50; income ■5,000	No category
8	Marks 61; attendance 71; income ■15,000	Merit-Based scholarship
9	Marks 60; attendance 90; income ■30,000	Need-Based scholarship
10	Marks 80; attendance 70; income ■8,000	No category

Note: Boundary values are included to check conditions and decision branches.

Activity 5: Conclusion

Multi-Stage Scholarship Shortlisting

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

6. Password Policy Validation Engine

1. Problem Statement

Evaluate a sample password using character-category and length checks, then report whether it satisfies the rules implemented in the supplied program.

2. Objectives

• Read a sample password.

• Check the password length.

• Check for uppercase and lowercase letters.

• Check for digits and special characters.

• Display whether the password is strong or identify missing requirements.

3. Assumptions

• This is a classroom demonstration only.

• Do not enter or evaluate a real personal password.

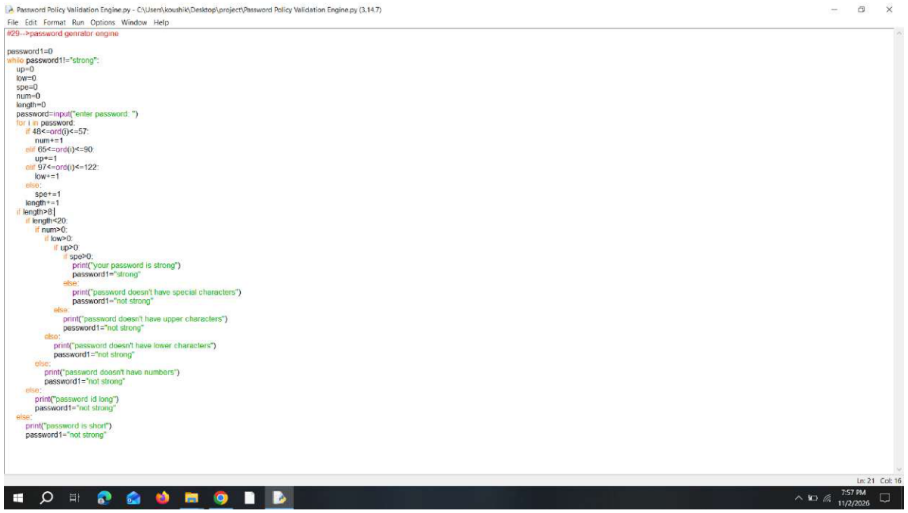
• The checks and thresholds are those implemented in the supplied program.

4. Algorithm

Read sample password → inspect characters and length → evaluate categories → display password strength result.

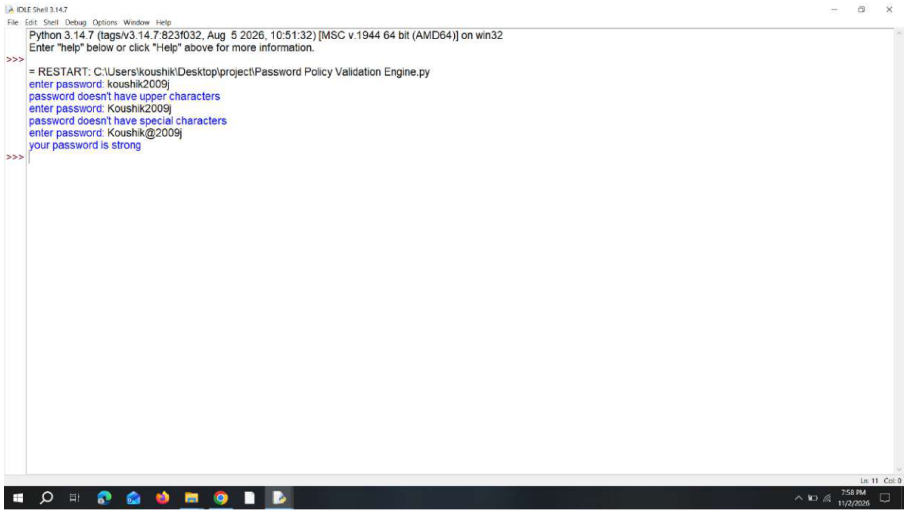
Activity 6: Password Policy Validation Engine

6. Original Python Program Screenshot



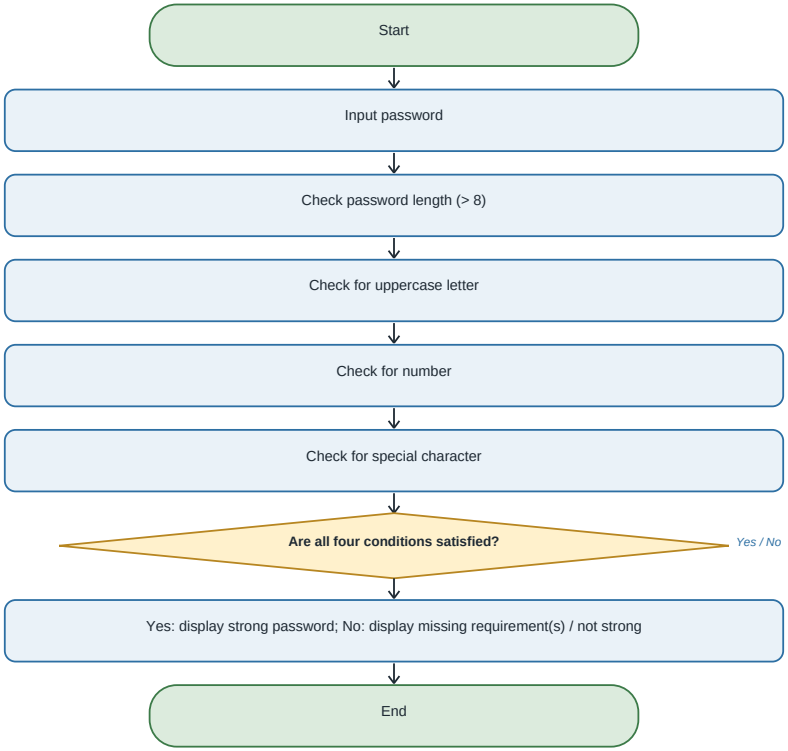
Activity 6: Password Policy Validation Engine

7. Original Output Screenshot



Activity 6: Flowchart

Password Policy Validation Engine



Activity 6: Pseudocode

Password Policy Validation Engine

```
START
INPUT password
SET strong = TRUE
IF length(password) <= 8 THEN
    DISPLAY password is short; strong = FALSE
CHECK whether password contains uppercase letter
IF absent THEN DISPLAY missing uppercase; strong = FALSE
CHECK whether password contains a number
IF absent THEN DISPLAY missing number; strong = FALSE
CHECK whether password contains a special character
IF absent THEN DISPLAY missing special character; strong = FALSE
IF strong THEN DISPLAY “password is strong”
ELSE DISPLAY “password is not strong”
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 6: Test Cases

Password Policy Validation Engine

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	Koushik@2009	Strong: length, uppercase, number and special character present
2	koushik2009j	Not strong: uppercase and special character missing
3	Koushik2009	Not strong: special character missing
4	Koushik@abcd	Not strong: number missing
5	koushik@2009	Not strong: uppercase missing
6	Ab@123	Not strong: password length ≤ 8
7	ABCDEFGH@1	Strong: all conditions satisfied
8	abcdefghi1@	Not strong: uppercase missing
9	ABCdefghi@	Not strong: number missing
10	123456789	Not strong: uppercase and special character missing

Note: Boundary values are included to check conditions and decision branches.

Activity 6: Conclusion

Password Policy Validation Engine

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

7. Progressive Electricity Billing

1. Problem Statement

Calculate electricity bills for multiple households using consumption slabs, a fixed charge, and identify the household with the highest bill.

2. Objectives

• Read the number of households.

• Read each household name and electricity units consumed.

• Calculate the energy charge using the program's consumption slabs.

• Add the fixed charge to obtain the total bill.

• Display household bills and identify the highest bill.

3. Assumptions

• The per-unit rates are defined in the supplied program.

• A fixed charge of Rs. 50 is added.

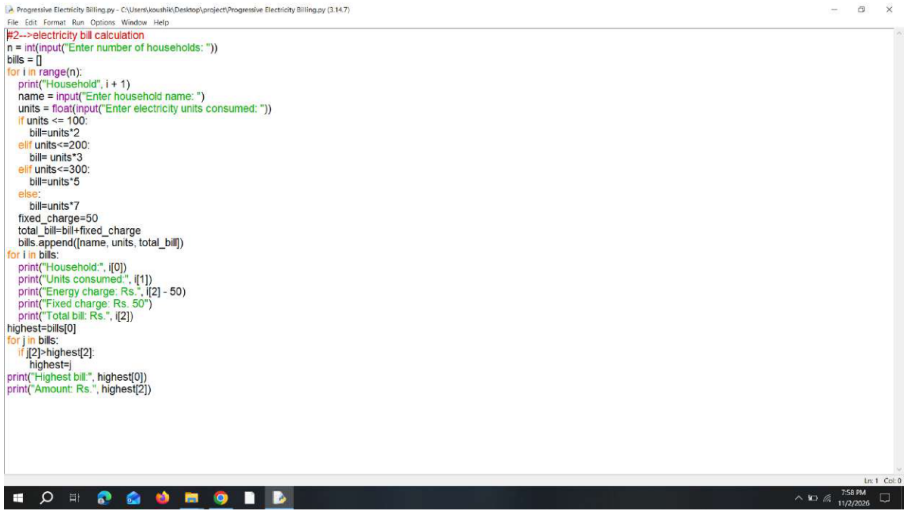
• This is an educational billing calculation, not an official utility tariff.

4. Algorithm

Read household usage → apply consumption slab rate → add fixed charge → store bill → find highest bill.

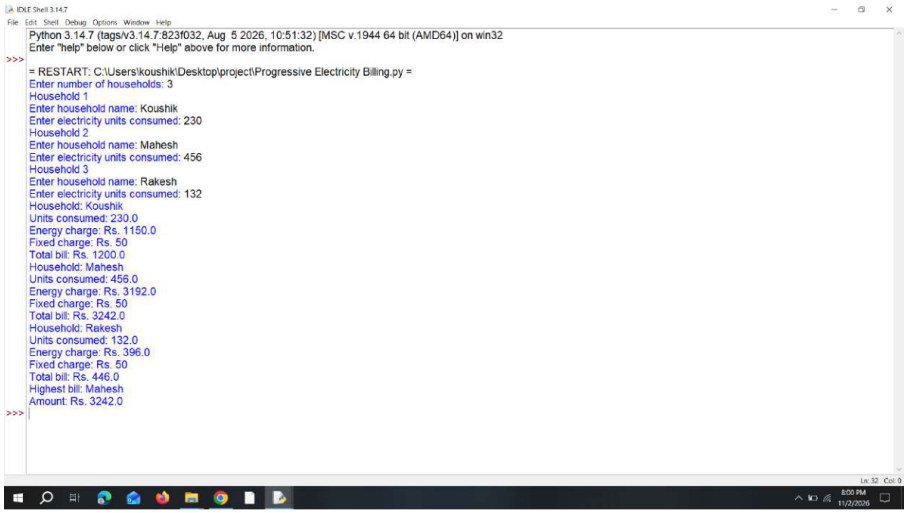
Activity 7: Progressive Electricity Billing

6. Original Python Program Screenshot



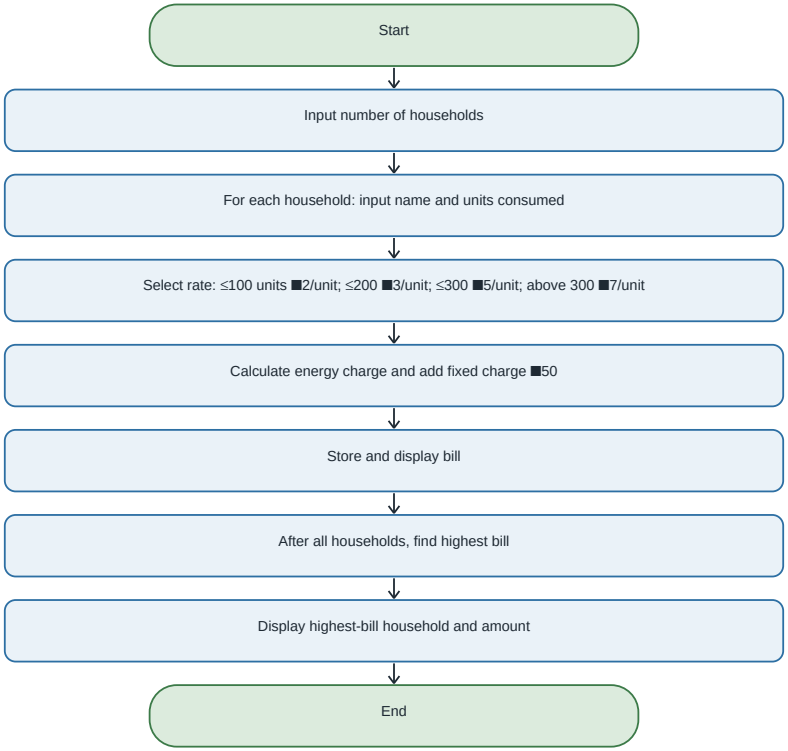
Activity 7: Progressive Electricity Billing

7. Original Output Screenshot



Activity 7: Flowchart

Progressive Electricity Billing



Activity 7: Pseudocode

Progressive Electricity Billing

```
START
INPUT n
SET bills = empty list
FOR each household
    INPUT name, units
    IF units <= 100 THEN rate = 2
    ELSE IF units <= 200 THEN rate = 3
    ELSE IF units <= 300 THEN rate = 5
    ELSE rate = 7
    energy_charge = units × rate
    total_bill = energy_charge + 50
    STORE name, units, total_bill
    DISPLAY bill details
END FOR
FIND household with maximum total_bill
DISPLAY highest bill details
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 7: Test Cases

Progressive Electricity Billing

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	Koushik; 230 units	Energy ■1,150; fixed ■50; total ■1,200
2	Mahesh; 456 units	Energy ■3,192; fixed ■50; total ■3,242
3	Rakesh; 132 units	Energy ■396; fixed ■50; total ■446
4	House A; 0 units	Energy ■0; total ■50
5	House B; 100 units	Energy ■200; total ■250
6	House C; 101 units	Energy ■303; total ■353
7	House D; 200 units	Energy ■600; total ■650
8	House E; 201 units	Energy ■1,005; total ■1,055
9	House F; 300 units	Energy ■1,500; total ■1,550
10	House G; 301 units	Energy ■2,107; total ■2,157

Note: Boundary values are included to check conditions and decision branches.

Activity 7: Conclusion

Progressive Electricity Billing

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

8. Smart Canteen Order and Stock Management

1. Problem Statement

Simulate a canteen order by checking menu items and updating the available stock after an order is placed.

2. Objectives

• Display the available food items and their stock.

• Read the customer's order.

• Check whether the requested item is supported and stock is available.

• Update stock after a valid order.

• Ask whether another item should be ordered.

3. Assumptions

• The menu and stock values are predefined in the supplied program.

• This is a classroom ordering simulation.

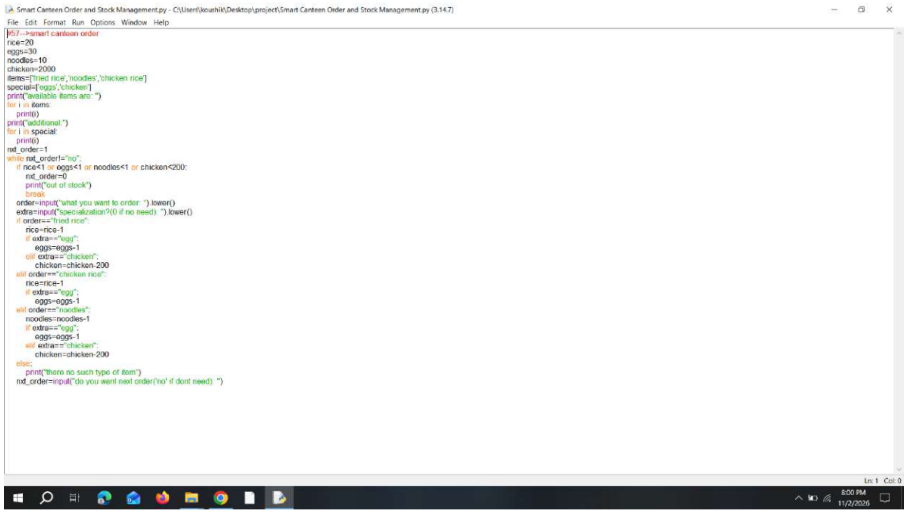
• Only the menu options handled by the code are supported.

4. Algorithm

Display menu and stock → read order → validate item and stock → update stock → ask for another order.

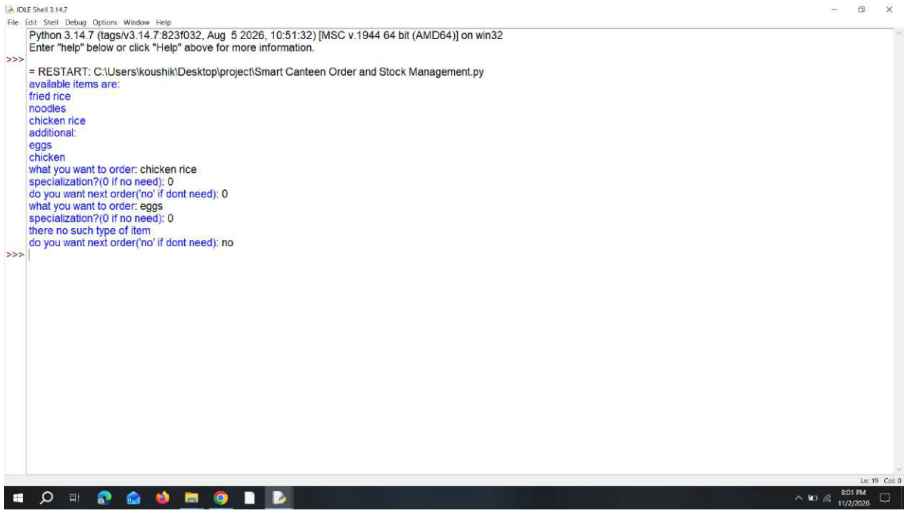
Activity 8: Smart Canteen Order and Stock Management

6. Original Python Program Screenshot



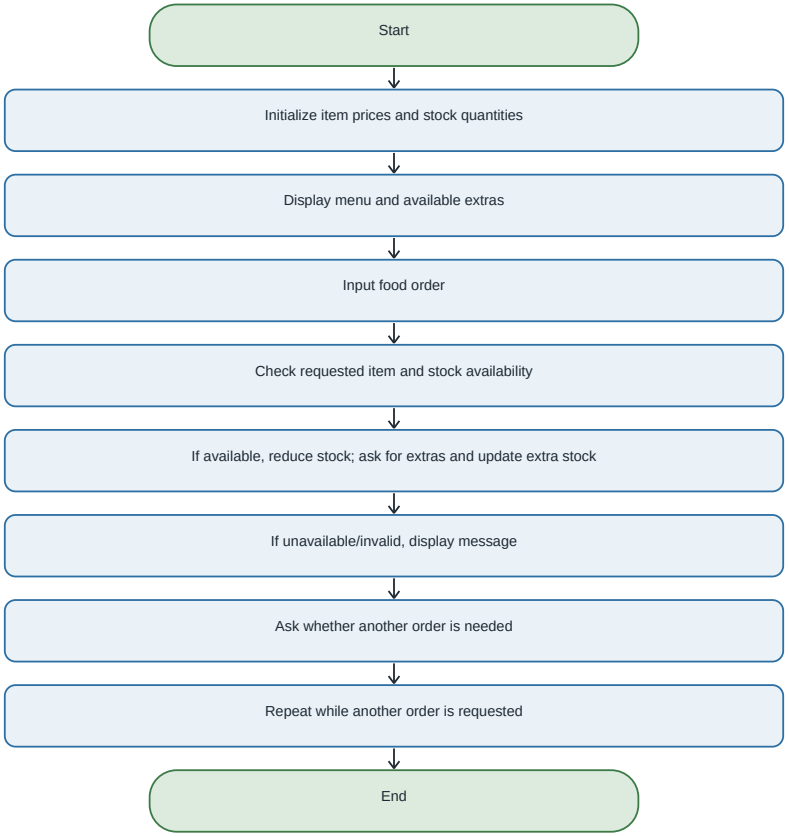
Activity 8: Smart Canteen Order and Stock Management

7. Original Output Screenshot



Activity 8: Flowchart

Smart Canteen Order and Stock Management



Activity 8: Pseudocode

Smart Canteen Order and Stock Management

```
START
INITIALIZE menu prices and stock: fried rice, noodles, chicken rice; eggs and chicken extras
DISPLAY available items and extras
SET next_order = “yes”
WHILE next_order is not “no”
    INPUT order
    INPUT extra
    IF requested item is valid AND stock is available THEN
        REDUCE corresponding item stock
        IF extra = egg THEN reduce egg stock
        ELSE IF extra = chicken THEN reduce chicken stock
        DISPLAY order status
    ELSE DISPLAY “there is no such type of item” or out of stock
    INPUT next_order
END WHILE
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 8: Test Cases

Smart Canteen Order and Stock Management

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	chicken rice; no extra; next=no	Valid menu item; stock reduced by one
2	fried rice; egg extra	Valid item; egg stock reduced by one
3	noodles; chicken extra	Valid item; chicken extra stock reduced
4	unknown item; next=no	Display invalid item message
5	eggs as main item; next=no	Display invalid item message (eggs are an extra)
6	repeat orders until stock becomes 0	Further order should report out of stock
7	fried rice; no extra; continue yes	Process first order and prompt for next order
8	chicken rice; egg extra; continue no	Process item and extra, then stop
9	noodles; egg extra; continue no	Process noodles and reduce egg stock
10	valid item after stock depleted	Display out-of-stock message

Note: Boundary values are included to check conditions and decision branches.

Activity 8: Conclusion

Smart Canteen Order and Stock Management

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

9. Smart Parking Fee Management

1. Problem Statement

Calculate a parking fee based on the vehicle type and parking duration, with a maximum duration and additional fee condition.

2. Objectives

• Display the supported vehicle types.

• Read the vehicle type and parking duration.

• Select the vehicle-specific maximum duration and base fee.

• Apply the program's fee calculation.

• Display the total fee or the extra-fee message.

3. Assumptions

• Vehicle types and fee values are predefined in the supplied program.

• The fee calculation follows the supplied code.

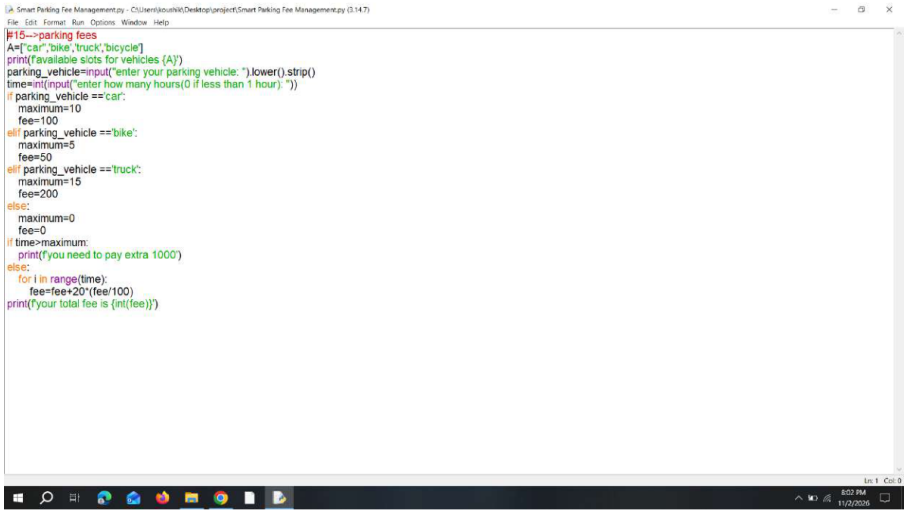
• This is a classroom parking-fee simulation.

4. Algorithm

Read vehicle and duration → select vehicle fee settings → check maximum duration → calculate fee → display result.

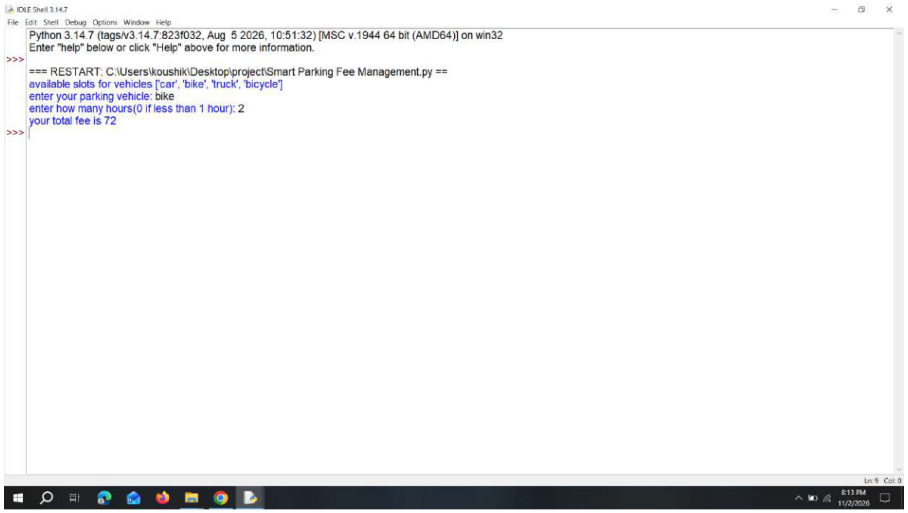
Activity 9: Smart Parking Fee Management

6. Original Python Program Screenshot



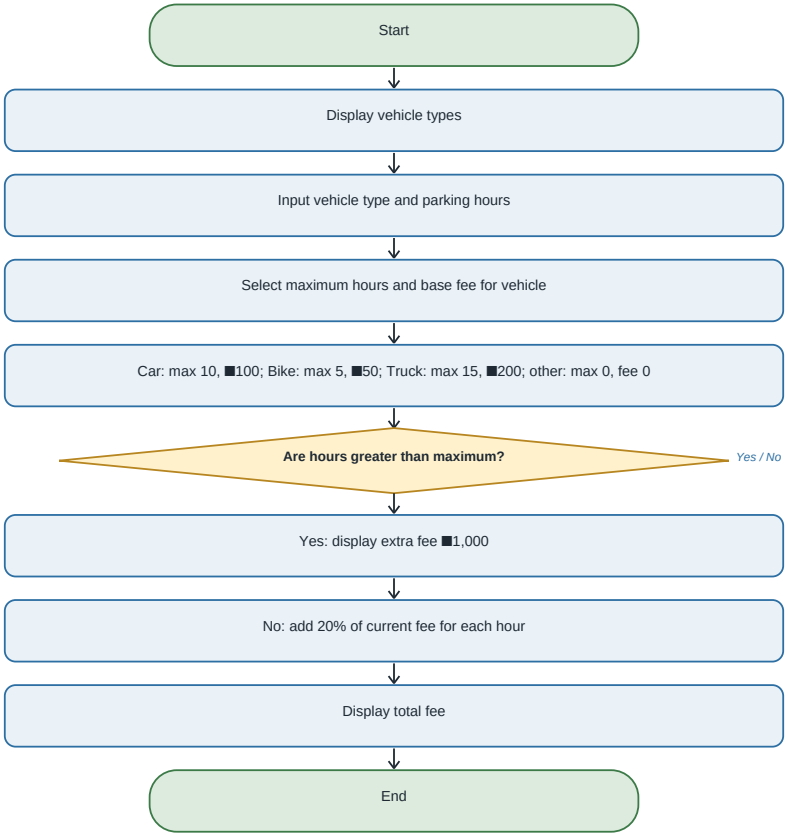
Activity 9: Smart Parking Fee Management

7. Original Output Screenshot



Activity 9: Flowchart

Smart Parking Fee Management



Activity 9: Pseudocode

Smart Parking Fee Management

```
START
DISPLAY vehicle types
INPUT vehicle_type, time
IF vehicle_type = car THEN maximum = 10; fee = 100
ELSE IF vehicle_type = bike THEN maximum = 5; fee = 50
ELSE IF vehicle_type = truck THEN maximum = 15; fee = 200
ELSE maximum = 0; fee = 0
IF time > maximum THEN DISPLAY extra fee ■1000
ELSE REPEAT time times: fee = fee + 20% of fee
DISPLAY integer fee
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 9: Test Cases

Smart Parking Fee Management

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	bike; 2 hours	■72
2	car; 0 hours	■100
3	car; 1 hour	■120
4	bike; 5 hours	■124 (rounded down from iterative fee)
5	bike; 6 hours	Display extra fee ■1,000
6	car; 10 hours	Calculate fee with 10 successive 20% increases
7	car; 11 hours	Display extra fee ■1,000
8	truck; 15 hours	Calculate fee with 15 successive increases
9	truck; 16 hours	Display extra fee ■1,000
10	bicycle; 2 hours	Vehicle falls into default branch (maximum 0; fee 0); hours exceed maximum

Note: Boundary values are included to check conditions and decision branches.

Activity 9: Conclusion

Smart Parking Fee Management

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.

10. University Scholarship Allocation

1. Problem Statement

Collect details for multiple students, identify students meeting the marks condition, and display the scholarship percentage based on income.

2. Objectives

• Read the number of students.

• Collect each student's name, marks, attendance, and income.

• Identify eligible students using the marks condition.

• Check each eligible student's income.

• Display the scholarship percentage assigned by the supplied program.

3. Assumptions

• Eligibility and income thresholds are those written in the supplied program.

• The program processes multiple students.

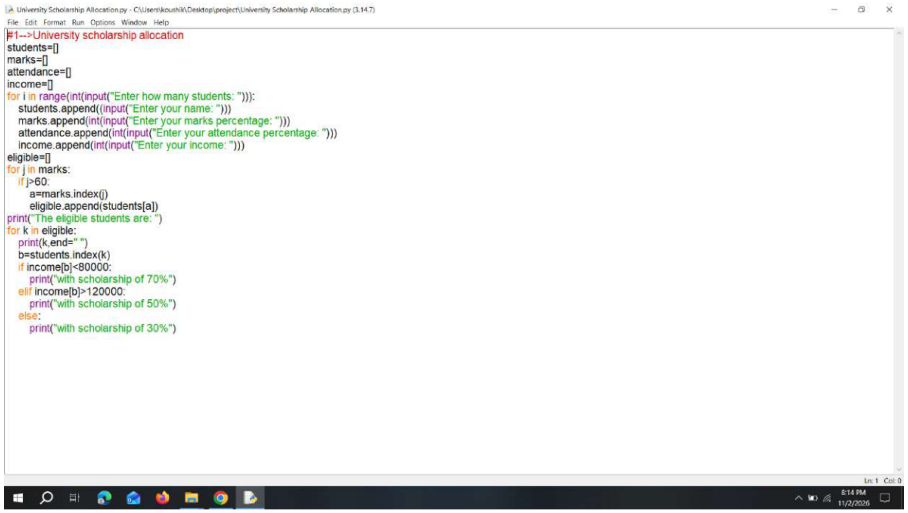
• Scholarship percentages are displayed as fixed categories.

4. Algorithm

Read student records → identify marks-eligible students → check income → assign scholarship percentage → display eligible students.

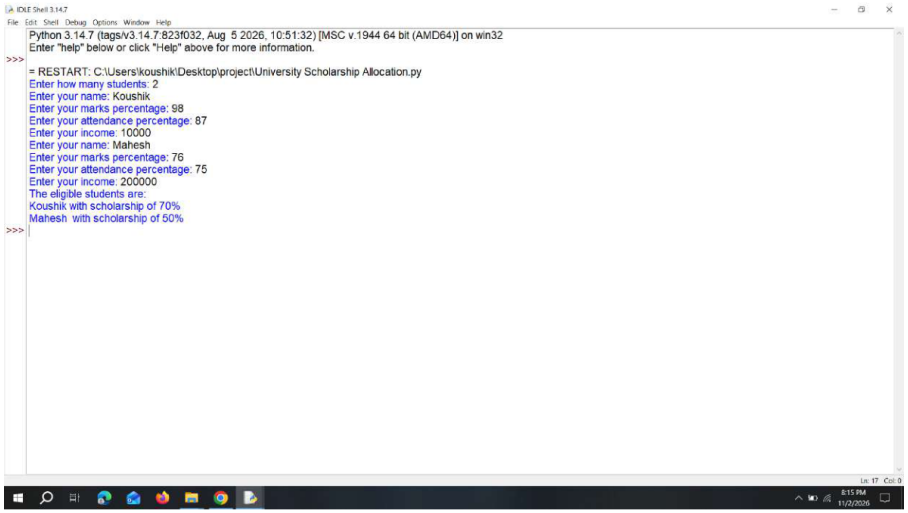
Activity 10: University Scholarship Allocation

6. Original Python Program Screenshot



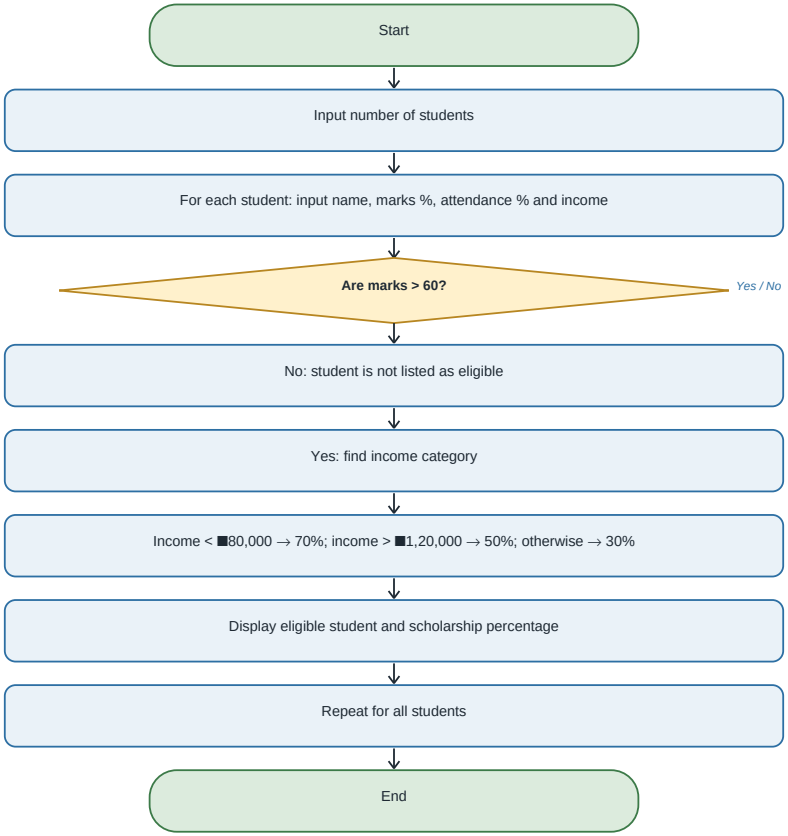
Activity 10: University Scholarship Allocation

7. Original Output Screenshot



Activity 10: Flowchart

University Scholarship Allocation



Activity 10: Pseudocode

University Scholarship Allocation

```
START
INPUT number_of_students
FOR each student
    INPUT name, marks, attendance, income
    STORE student details
END FOR
DISPLAY eligible students
FOR each student
    IF marks > 60 THEN
        IF income < 80000 THEN scholarship = 70%
        ELSE IF income > 120000 THEN scholarship = 50%
        ELSE scholarship = 30%
        DISPLAY name and scholarship
END FOR
STOP
```

Pseudocode is a language-independent description of the program logic.

Activity 10: Test Cases

University Scholarship Allocation

10 representative test cases based on the supplied program logic

TC	Input / Test Data	Expected Result
1	Koushik; marks 98; attendance 87; income ■10,000	Eligible; 70% scholarship
2	Mahesh; marks 76; attendance 75; income ■2,00,000	Eligible; 50% scholarship
3	A; marks 60; attendance 90; income ■50,000	Not listed as eligible
4	B; marks 61; attendance 50; income ■80,000	Eligible; 30% scholarship
5	C; marks 80; attendance 80; income ■1,20,000	Eligible; 30% scholarship
6	D; marks 90; attendance 90; income ■1,20,001	Eligible; 50% scholarship
7	E; marks 90; attendance 90; income ■79,999	Eligible; 70% scholarship
8	F; marks 60; attendance 60; income ■2,00,000	Not listed as eligible
9	Two students: marks 75 and 55	Only first student is eligible
10	Student count = 1; marks 61; income ■90,000	Eligible; 30% scholarship

Note: Boundary values are included to check conditions and decision branches.

Activity 10: Conclusion

University Scholarship Allocation

This activity demonstrates how Python input, variables, conditional statements, loops, lists, and calculations can be used to solve a practical problem. The program processes the entered information according to the rules shown in the supplied code and displays the result.